

REST API. Products and Markets

- Methods
 - Limits and Headers
- Add productss
 - Entities
 - Products
 - Product
 - Category
 - Translation
 - ProductRemovePatch
 - Product Identity and Tracking Correspondence
 - Update logic detailed description
 - Scalar Fields
 - Array Fields
 - Translations
 - Clearable Fields
 - Update Examples
- Delete product
 - Entities
 - Products
 - Product
 - Delete Examples
- Add markets
 - Entities
 - Markets
 - Market
 - Product
 - MarketProductRemovePatch
 - Add markets Update Semantics
 - URL Language Rule
 - Add markets Examples
 - CSV Feed Mapping
- Response Format
 - Entities
 - Response object
 - Item
 - Add productss (POST/DELETE v1/products)
 - Add markets (POST v1/markets)
 - Response Codes
 - Error Codes

Methods

Method	Type	Resource	Input	Output
Add productss	POST	/v1/producs	[Products]	
	T	ts		

Delete products	DELETE	/v1/products		
Add markets	POST	/v1/markets	[Markets]	

Limits and Headers

Limits

Limit	Value
Max items per request	1,000
Max body size	10 MB
Max requests per minute per siteId	60
Validation latency target	< 5s
Reflection in recommendations target	< 30s

Headers

Content-Type: application/json

Add products

Add products API is a normalized product-ingestion API for ecommerce app and plugin developers.

It is designed for integrations that stream product data directly from platforms such as Shopify, WooCommerce, Horoshop, and similar systems, instead of maintaining a custom product-feed export for import.

Endpoint POST /v1/products

The endpoint must do the following:

- Via this method catalog, translations, base commercial data can be created and updated
 - An initial import of the current catalog state can be made

- `create` / `update` action must be specified in a separate parameter to show needed action
 - Each submitted product item is processed independently by its `action`
 - `create` creates a new product and requires the full create payload
 - `update` is intended for existing products
- A single request can contain up to `1,000` items. Items are validated independently. If one fails, the rest may still be accepted

Entities

Products

```

1 {
2   "siteId": "my_store",
3   "languageCode": "uk",
4   "languageChanged": false,
5   "products": [ Product ]
6 }
```

Parameter	Type	Required	Description
<code>siteId</code>	String	Yes	Client site/account identifier. Max <code>128</code> chars. Allowed characters: letters, digits, <code>.</code> , <code>_</code> , <code>-</code> .
<code>languageCode</code>	String	Yes	Default language for top-level product fields in this request. Must be a BCP 47 language tag, for example <code>"uk"</code> , <code>"en"</code> , <code>"de"</code> , <code>"en-GB"</code> . Top-level <code>name</code> , <code>description</code> , <code>url</code> , and <code>categories</code> are in this language. The server records the <code>languageCode</code> from the first accepted <code>Add products</code> request for each <code>siteId</code> as the site default. Subsequent <code>Add products</code> requests must use the same <code>languageCode</code> unless <code>languageChanged</code> is <code>true</code> .
<code>languageChanged</code>	Boolean	No	Must be set to <code>true</code> when intentionally changing the default language for this <code>siteId</code> . If <code>languageCode</code> differs from the stored site default and <code>languageChanged</code> is not <code>true</code> , the entire request is rejected with <code>LANGUAGE_CODE_MISMATCH</code> . Default: <code>false</code> . If <code>languageChanged</code> is <code>true</code> and the submitted <code>languageCode</code> matches the stored site default, the flag is treated as a no-op.

products	Array	Yes	Array of Product .
----------	-------	-----	--------------------

Product

```
1  {
2    "action": "create/update",
3    "productId": "shopify_variant_987",
4    "updatedAt": "2026-04-01T12:00:00Z",
5    "name": "Men's T-Shirt / Black / M",
6    "url": "https://mystore.ua/en/products/tshirt-black-m",
7    "imageUrl": "https://cdn.mystore.ua/images/tshirt-black-m.jpg",
8    "isInStock": 1,
9    "price": 599,
10   "currency": "UAH",
11   "description": "descriptpion",
12   "categories": [ Category ],
13   "translations": [ Translation ],
14   "tags": {
15     "Color": ["Black"],
16     "Size": ["M"]
17   },
18   "remove" : ProductRemovePatch
19 }
```

Parameter	Type	Required	Description
action	String	Yes	"create" , "update"
productId	String	Yes	Unique product identifier within siteId . Must be the same identifier used in tracking events, recommendation requests, and market data. Max 128 chars.
updatedAt	String	Yes	Source ordering timestamp for this item. Must be RFC3339 / ISO 8601 in UTC, for example 2026-04-01T09:07:15Z . Used to ignore stale out-of-order events during asynchronous processing. Prefer the source object update time when available. If unavailable, use the webhook event timestamp or the time the change was detected by the plugin.
name	String	Yes on create	Display name in default language. Max 500 chars.
imageUrl	String	Yes on create	Main image URL. Must be a valid absolute URL. Minimum image size 200x200 .
isInStock	Integer	Yes on create	1 = in stock, 0 = out of stock in the base commercial state used when no market override is present.

url	String	Yes on create	Product URL for default language. Must be a valid absolute URL.
currency	String	Yes on create	ISO 4217 currency code for the base commercial state, for example "UAH" , "USD" , "EUR" .
price	Float	Yes on create	Selling price in the base commercial state used when no market override is present. No currency symbol.
categories	Array	Yes on create	Array of Category . Minimum 1 element. Names and paths are in default language. On update: provided value replaces all categories. Omitted value preserves existing categories.
itemGroupId	String	No	Shared family/group identifier used to group product variants. For variant-based tracking, multiple variant productId values may share the same itemGroupId . Max 128 chars.
oldPrice	Float	No	Original price in the base commercial state. If oldPrice and price are both present with numeric values in the same submitted item, oldPrice must be greater than price .
brand	String	No	Brand or manufacturer. Max 256 chars.
description	String	No	Plain text in default language. UTF-8 string. Max 10,000 chars. HTML is not supported.
translations	Array	No	Array of Translation .Dictionary of language code → translation object. Example keys: "en", "de", "pl".
tags	Object	No	Product tags. On update: provided value overwrites existing value, omitted value preserves existing value.
remove	Object	No	Explicit remove instructions for product update operations. Use this object to remove existing optional product data.

Category

```

1 {
2   "id": "8",
3   "name": "Футболки",

```

```

4  "path": ["Одяг", "Верх", "Футболки"],
5  "type": "category/collection"
6  }

```

Parameter	Type	Required	Description
id	String	Conditional	Required if name is not provided.
name	String	Conditional	Required if id is not provided. In default language.
path	Array	No	Hierarchy root to leaf. Max 10 levels. In default language.
type	String	No	"category" — taxonomy/product type. Hierarchical. Used for category-based recommendations and content filtering. "collection" — merchandising group. Flat. Used for cross-sell and filtering. Default: "category" if omitted.

Translation

```

1  {
2    "en": {
3      "name": "Men's T-Shirt",
4      "url": "https://mystore.ua/en/tshirt",
5      "description": "description",
6      "categories": [ Category ]
7    }
8  }

```

Parameter	Type	Required	Description
name	String	No	Localized name.
description	String	No	Localized plain-text description. UTF-8 string. Max 10,000 chars. HTML is not supported.
url	String	No	Localized URL. Must be a valid absolute URL.
categories	Array	No	Array of Category . Localized category names and paths. Category id stays the same.

ProductRemovePatch

```

1  {
2    "fields": ["price"],

```

```

3     "tags": ["Size"],
4     "translations":["en"]
5 }

```

Parameter	Type	Required	Description
fields	Array	No	Product fields to remove. Allowed values depend on the endpoint context.
tags	Array	No	Tag names to remove.
translations	Array	No	Language codes of translations to remove.

- After the first accepted request with `languageChanged: true`, the new `languageCode` becomes the stored site default for that `siteId`.
- The server does not automatically migrate content between languages.
- The integration is responsible for sending a full catalog re-sync and correct `translations` structure.
- Until the re-sync completes, the catalog may contain products with mixed stored default languages, and recommendation rendering serves stored content as-is.
- `languageCode` should remain consistent across all `Add products` requests for the same `siteId`. Changing it is a site-level configuration change, not a per-request content choice.

If a `Add products` request sends a `languageCode` that differs from the stored site default and `languageChanged` is not `true`, the entire request is rejected with `409 Conflict` and `LANGUAGE_CODE_MISMATCH`.

Example mismatch response:

```

1 {
2   "errorCode": "LANGUAGE_CODE_MISMATCH",
3   "message": "Request languageCode 'en' does not match site default 'uk'. Set languageChanged to true",
4   "storedLanguage": "uk",
5   "receivedLanguage": "en"
6 }

```

Request example

```
1 {
2   "siteId": "my_store",
3   "languageCode": "uk",
4   "products": [
5     {
6       "action": "create",
7       "productId": "shopify_variant_987",
8       "updatedAt": "2026-04-01T09:07:15Z",
9       "itemGroupId": "shopify_product_123",
10      "name": "Футболка чоловіча / Чорний / М",
11      "description": "Текстовий опис",
12      "url": "https://mystore.ua/products/futbolka-cholovicha-black-m",
13      "imageUrl": "https://cdn.mystore.ua/images/futbolka-cholovicha-bl",
14      "brand": "MyBrand",
15      "isInStock": 1,
16      "price": 599.00,
17      "oldPrice": 699.00,
18      "currency": "UAH",
19      "categories": [
20        {
21          "id": "8",
22          "name": "Футболки",
23          "path": ["Одяг", "Верх", "Футболки"],
24          "type": "category"
25        },
26        {
27          "id": "305",
28          "name": "Літній розпродаж",
29          "type": "collection"
30        }
31      ],
32      "tags": {
33        "Color": ["Black"],
34        "Size": ["M"]
35      }
36      "translations": [
37        {
38          "en": {
39            "name": "Men's T-Shirt / Black / M",
40            "description": "string",
41            "url": "https://mystore.ua/en/products/tshirt-black-m",
42            "categories": [
43              {
44                "id": "8",
45                "name": "T-Shirts",
46                "path": ["Clothing", "Tops", "T-Shirts"]
47              }
48            ]
49          }
50        }
51      ]
52    }
53  }
```

Product Identity and Tracking Correspondence

`productId` must match the entity identifier used by the client tracking setup.

- If user behavior is tracked on variant IDs, then `productId` must be the variant ID.
- If user behavior is tracked on parent product IDs, then `productId` must be the parent product ID.

- The same `siteId` must use one consistent `productId` strategy across `Add products`, `Add markets`, tracking, and recommendation requests.
- For variant-based tracking, `itemGroupId` should contain the shared parent or family identifier used to group variants.

Update logic detailed description

- Products not included in the request are preserved unchanged
- If a `create` item targets an existing product, it is processed using update semantics, but all create-required fields must still be present
- `update` applies only to the submitted product and may contain only changed fields, plus required update fields
- No existence check is performed during request validation

Examples of valid partial business updates:

- update stock or price for only 2 products → send only those 2 product items; all other products remain unchanged
- change only one tag for 1 product → send only that `tags.*` field for that product
- add or replace one translation for 1 product → send only that locale inside `translations`

Scalar Fields

- Omitted field → preserved unchanged
- New value → overwritten
- Use `remove.fields` for fields removing

Array Fields

- Omitted `categories` → preserved unchanged
- New `categories` array → full replacement of categories
- `categories` cannot be set to `null`
- Omitted `tags.*` → preserved unchanged for that tag
- New `tags.*` array → full replacement of that tag only
- Use `remove.tags` for fields removing

Translations

- Omit `translations` → all translations preserved
- Use `remove.fields:["translations"]` for removing all translations removed

- `translations: [{ "en": { ... } }]` → English translation fully replaced, other languages preserved
- Use. `remove.translations: ["en"]` → English translation removed, other languages preserved

Important:

- Locale objects inside `translations` use replacement semantics, not field-level patch semantics.
- This means `translations: [{ "en": { "name": "..."} }]` replaces the English translation object with only the submitted fields.
- If a field such as English `description` or `categories` existed before and is omitted in the new English object, it is removed from that locale.

Clearable Fields

Field	Clearable	How
<code>oldPrice</code>	Yes	Send <code>remove.fields: ["oldPrice"]</code>
<code>description</code> , <code>brand</code> , <code>itemGroupId</code>	Yes	Send <code>remove.fields: ["description", "brand"]</code>
<code>tag{name}</code>	Yes	Send <code>remove.tags: [{"name}"]</code>
<code>translations</code>	Yes	<code>remove.fields: ["translations"]</code> clears all, <code>remove.translations: ["en"]</code> clears one
<code>name</code> , <code>url</code> , <code>image_url</code> , <code>price</code> , <code>currency</code> , <code>isInStock</code> , <code>categories</code> , <code>updatedAt</code> , <code>discount</code>	No	Required, ignored, or non-clearable

Update Examples

Update price for one product. All other products remain unchanged:

```
{
  "siteId": "my_store",
  "languageCode": "uk",
  "products": [
    {
```

```

6      "action": "update",
7      "productId": "430738",
8      "updatedAt": "2026-04-01T10:07:00Z",
9      "price": 47999.00,
10     "oldPrice": 54999.00
11   }
12 ]
13 }

```

Update only one tag and preserve all other tags:

```

1  {
2    "siteId": "my_store",
3    "languageCode": "uk",
4    "products": [
5      {
6        "action": "update",
7        "productId": "430738",
8        "updatedAt": "2026-04-01T10:07:30Z",
9        "tags":{"Color": ["Black"]}
10     }
11   ]
12 }

```

Remove one tag:

```

1  {
2    "siteId": "my_store",
3    "languageCode": "uk",
4    "products": [
5      {
6        "action": "update",
7        "productId": "430738",
8        "updatedAt": "2026-04-01T10:07:45Z",
9        "remove": {"tags":["Season"]}
10     }
11   ]
12 }

```

Remove old price. Discount is derived by the server and does not need to be sent:

```

1  {
2    "siteId": "my_store",
3    "languageCode": "uk",
4    "products": [
5      {
6        "action": "update",
7        "productId": "430738",
8        "updatedAt": "2026-04-01T10:07:45Z",
9        "price": 599.00,
10     "remove": {"fields":["oldPrice"]}
11   }
12 ]
13 }

```

Update stock only for one product:

```

1  {
2    "siteId": "my_store",
3    "languageCode": "uk",
4    "products": [
5      {

```

```

6     "action": "update",
7     "productId": "430738",
8     "updatedAt": "2026-04-01T10:08:30Z",
9     "isInStock": 0
10  }
11  ]
12 }

```

Add German translation and preserve other locales:

```

1  {
2    "siteId": "my_store",
3    "languageCode": "uk",
4    "products": [
5      {
6        "action": "update",
7        "productId": "430738",
8        "updatedAt": "2026-04-01T10:08:45Z",
9        "translations": [{
10         "de": {
11           "name": "Herren T-Shirt",
12           "url": "https://mystore.ua/de/tshirt"
13         }
14       }]
15     }
16   ]
17 }

```

Replace one locale completely. Omitted fields inside `en` are removed from that locale:

```

1  {
2    "siteId": "my_store",
3    "languageCode": "uk",
4    "products": [
5      {
6        "action": "update",
7        "productId": "430738",
8        "updatedAt": "2026-04-01T10:08:50Z",
9        "translations": [{
10         "en": {
11           "name": "Men's T-Shirt",
12           "url": "https://mystore.ua/en/tshirt"
13         }
14       }]
15     }
16   ]
17 }

```

Remove one translation:

```

1  {
2    "action": "update",
3    "productId": "430738",
4    "updatedAt": "2026-04-01T10:09:00Z",
5    "remove": {
6      "translations": ["en"]
7    }
8  }

```

Remove all translations:

```

1  {
2    "action": "update",

```

```

3  "productId": "430738",
4  "updatedAt": "2026-04-01T10:09:30Z",
5  "remove": {
6    "fields": ["translations"]
7  }
8  }

```

Language-change request:

If the default language intentionally changes, for example from `uk` to `en` :

1. Set `languageChanged: true` .
2. Submit the first re-sync batch with the new `languageCode` . After this request is accepted, the stored site default updates immediately. Subsequent re-sync batches use the new `languageCode` without `languageChanged` .
3. Re-sync the full catalog with complete product payloads in the new default language, with `translations` restructured so the old default language moves into `translations` when needed.
4. Re-sync `Add markets` if URL locale keys were added, removed, or changed.

```

1  {
2    "siteId": "my_store",
3    "languageCode": "en",
4    "languageChanged": true,
5    "products": [
6      {
7        "action": "create",
8        "productId": "shopify_variant_987",
9        "updatedAt": "2026-04-01T12:00:00Z",
10       "name": "Men's T-Shirt / Black / M",
11       "url": "https://mystore.ua/en/products/tshirt-black-m",
12       "imageUrl": "https://cdn.mystore.ua/images/tshirt-black-m.jpg",
13       "isInStock": 1,
14       "price": 599.00,
15       "currency": "UAH",
16       "categories": [
17         {
18           "id": "8",
19           "name": "T-Shirts",
20           "path": ["Clothing", "Tops", "T-Shirts"]
21         }
22       ],
23       "translations": [
24         {
25           "uk": {
26             "name": "Футболка чоловіча / Чорний / М",
27             "url": "https://mystore.ua/products/futbolka-cholovicha-black-m"
28           }
29         }
30       ]
31     }
32   ]
33 }

```

Delete product

Endpoint

DELETE /v1/products

The endpoint must do the following:

- Via this method products can be deleted
 - Each submitted product item is processed independently
- A single request can contain up to 1,000 items. Items are validated independently. If one fails, the rest may still be accepted
- Requests are validated synchronously. Accepted items may be processed asynchronously
- productId, updatedAt are required
- Idempotent no-op if the product is already absent. updatedAt remains required to protect against stale deletes. If the source platform does not provide a product-level timestamp for delete, use the webhook event timestamp or the time deletion was detected by the plugin.

Entities

Products

```
1 {  
2   "siteId": "my_store",  
3   "products": [ Product ]  
4 }
```

Parameter	Type	Required	Description
siteId	String	Yes	Client site/account identifier. Max 128 chars. Allowed characters: letters, digits, ., _, -.
products	Array	Yes	Array of Product.

Product

```
1 {  
2   "productId": "430738",  
3   "updatedAt": "2026-04-01T10:10:00Z"  
4 }
```

Parameter	Type	Required	Description
productId	String	Yes	Unique product identifier within siteId . Must be the same identifier used in tracking events, recommendation requests, and market data. Max 128 chars.
updatedAt	String	Yes	Source ordering timestamp for this item. Must be RFC3339 / ISO 8601 in UTC, for example 2026-04-01T09:07:15Z . Used to ignore stale out-of-order events during asynchronous processing. Prefer the source object update time when available. If unavailable, use the webhook event timestamp or the time the change was detected by the plugin.

Delete Examples

Delete one product:

```

1 {
2   "siteId": "my_store",
3   "products": [
4     {
5       "productId": "430738",
6       "updatedAt": "2026-04-01T10:07:00Z"
7     }
8   ]
9 }
```

Add markets

Market-specific prices, stock, and URLs

The method will be used for stores operating in multiple markets or regions with different prices and availability.

Update model: product-level patch stream. productId and updatedAt are required.

Market fields are patch-based.

Important:

- markets may contain any subset of products. Products not included in the request are not changed.
- Each submitted market product updates only the submitted fields for the same siteId + marketId + productId .

- Fields omitted in a submitted market item are preserved unchanged.

If a product is deleted through `v1/products` , all its market data is removed automatically.

To remove a product from a market in v1.1, send `isInStock: 0` .

Endpoint

POST `/v1/markets`

Entities

Markets

```
1 {  
2   "siteId": "my_store",  
3   "markets": [ Market ]  
4 }
```

Parameter	Type	Required	Description
siteId	String	Yes	Client site/account identifier. Max 128 chars. Allowed characters: letters, digits, . , _ , - .
markets	Array	Yes	Array of Market .

Market

```
1 {  
2   "marketId": "ca",  
3   "products": [ Product ]  
4 }
```

Parameter	Type	Required	Description
marketId	String	Yes	Market identifier. Can be country code, city code, Shopify Market handle, or another client-defined market key. Must match marketId used in recommendation requests. Max 128 chars. Allowed characters: letters, digits, . , _ , - .
products	Array	Yes	Array of Product with market-specific properties.

Product

```
1 {
2   "productId": "SKU123",
3   "updatedAt": "2026-04-01T09:07:15Z",
4   "price": 29.99,
5   "oldPrice": 39.99,
6   "currency": "CAD",
7   "isInStock": 1,
8   "urls": {
9     "en": "https://example.ca/en-ca/sku123",
10    "fr": "https://example.ca/fr-ca/sku123"
11  },
12  "remove": MarketProductRemovePatch
13 }
```

Parameter	Type	Required	Description
productId	String	Yes	Must use the same identity strategy as <code>Add products</code> and tracking. No base-product existence check is performed during request validation.
updatedAt	String	Yes	Source ordering timestamp for this market item. Must be RFC3339 / ISO 8601 in UTC. Used to ignore stale market updates during asynchronous processing. Prefer the source object update time when available. If unavailable, use the webhook event timestamp or the time the change was detected by the plugin.
price	Float	No	Market-specific selling price. Omit to preserve existing value. Sending <code>null</code> is invalid.
currency	String	No	ISO 4217 currency code. Omit to preserve existing value. Sending <code>null</code> is invalid.
isInStock	Integer	No	<code>1</code> or <code>0</code> . Market-specific availability. Omit to preserve existing value. Sending <code>null</code> is invalid. Use <code>0</code> to remove the product from recommendations in this market.
oldPrice	Float	No	Market-specific original price. Omit to preserve existing value. Send <code>null</code> to explicitly mark that there is no <code>oldPrice</code> in this market. Base <code>oldPrice</code> must not be used for this market when <code>oldPrice</code> is explicitly cleared. If <code>oldPrice</code> and <code>price</code> are both present with numeric values in the same submitted item, <code>oldPrice</code> must be greater than <code>price</code> .

urls	Object	No	Dictionary of language code → market-specific URL. Omit to preserve existing URLs. Send <code>null</code> to clear all. Submitted locale keys update only the submitted products' locales. Submitted URL values must be valid absolute URLs.
------	--------	----	--

MarketProductRemovePatch

```

1 {
2   "fields": ["price"],
3   "urlsTranslations": ["en"]
4 }
```

Add markets Update Semantics

Add `markets` is a product-level patch stream, not request-level replacement.

Rules:

- Products not included in the request are preserved unchanged.
- Integrations should send `Add products` first, then `Add markets`.
- `Add markets` does not check whether the base product already exists during request validation.
- Market data may be accepted and stored before the corresponding `Add products` exists.
- Market data is effective in recommendation resolution only after the matching base product exists.
- For each submitted `marketId + productId`, only `productId` and `updatedAt` are always required.
- Each submitted market item must also include at least one market field to change: `price`, `currency`, `isInStock`, `oldPrice`, or `urls`.
- All market fields use patch semantics:
 - omit `price` → preserve existing `price`
 - send `price: null` → invalid
 - omit `currency` → preserve existing `currency`
 - send `currency: null` → invalid
 - omit `isInStock` → preserve existing `isInStock`
 - send `isInStock: null` → invalid

- omit `oldPrice` → preserve existing `oldPrice`
- send `remove.fields:["oldPrice"]` → explicitly mark that this market has no `oldPrice`; base `oldPrice` must not be used for this market
- omit `urls` → preserve existing market-specific URLs
- send `remove.fields:["urls"]` → clear all market-specific URLs
- send `urls: { "en": "..."}` → upsert only locale `en`
- send `remove.urlsTranslations: ["en"]` → clear only locale `en`
- When market-specific URLs are cleared or absent, recommendation rendering falls back to translation URL or base product URL.
- A market record becomes effective in recommendation resolution only when:
 - the matching base product exists
 - the effective market record has at minimum `price`, `currency`, and `isInStock`
- Incomplete market records may be stored, but they are not used in recommendations until the minimum required market fields are available.

Market discount note:

- `discount` is not sent in `Add markets`.
- If needed by downstream logic, market discount is derived from `oldPrice` and `price`.
- To remove a market discount, send `remove.fields:["oldPrice"]`.

URL Language Rule

Keys in `urls` must use BCP 47 language tags and should correspond to a language already defined in the product:

- the default `languageCode` from `Add products`
- or a language key from the product `translations`

If `urls` references a language not present in product translations, the request may still be accepted. Recommendation rendering falls back to default-language content and uses the provided market URL.

Examples of valid partial business updates:

- update stock for only 3 products in market `ca` → send only those 3 product items with `isInStock`; all other market fields are preserved
- update price for 1 product in market `ua` → send only that product item with changed price fields; stock and URLs are preserved

- remove old price / discount for 1 product → send the product item with `remove.fields:`
`["oldPrice"]`

Add markets Examples

Update stock only for one product. Existing market price and currency are preserved:

```
1 {
2   "siteId": "my_store",
3   "markets": [
4     {
5       "marketId": "ca",
6       "products": [
7         {
8           "productId": "SKU123",
9           "updatedAt": "2026-04-01T10:15:00Z",
10          "isInStock": 0
11        }
12      ]
13    }
14  ]
15 }
```

Update price only for one product. Existing stock is preserved:

```
1 {
2   "siteId": "my_store",
3   "markets": [
4     {
5       "marketId": "ua",
6       "products": [
7         {
8           "productId": "SKU123",
9           "updatedAt": "2026-04-01T10:20:00Z",
10          "price": 120.00,
11          "currency": "UAH"
12        }
13      ]
14    }
15  ]
16 }
```

Remove market old price and market discount. This explicitly means there is no `oldPrice` in this market:

```
1 {
2   "siteId": "my_store",
3   "markets": [
4     {
5       "marketId": "ua",
6       "products": [
7         {
8           "productId": "SKU123",
9           "updatedAt": "2026-04-01T10:25:00Z",
10          "remove": { "fields": ["oldPrice"] }
11        }
12      ]
13    }
14  ]
15 }
```

Update only some products in one market. Products not listed remain unchanged:

```
1 {
2   "siteId": "my_store",
3   "markets": [
4     {
5       "marketId": "ca",
6       "products": [
7         {
8           "productId": "SKU123",
9           "updatedAt": "2026-04-01T10:30:00Z",
10          "price": 29.99,
11          "currency": "CAD",
12          "isInStock": 1
13        },
14        {
15          "productId": "SKU456",
16          "updatedAt": "2026-04-01T10:31:00Z",
17          "price": 49.00,
18          "currency": "CAD",
19          "isInStock": 0
20        }
21      ]
22    }
23  ]
24 }
```

Clear market-specific URLs and fall back to base product URLs

```
1 {
2   "siteId": "my_store",
3   "markets": [
4     {
5       "marketId": "ca",
6       "products": [
7         {
8           "productId": "SKU123",
9           "updatedAt": "2026-04-01T10:35:00Z",
10          "remove": { "fields": ["urls"] }
11        }
12      ]
13    }
14  ]
15 }
```

If `salePrice` equals `price` or is absent, send only `price` .

CSV Feed Mapping

CSV Field	API Field
id	productId
marketId	marketId
availability	isInStock
price	oldPrice

salePrice	price
currency	currency

Response Format

Every request returns a request-level envelope plus item-level results.

Important:

- Item results describe synchronous validation and queue-publishing outcomes only.
- `accepted` means the item passed synchronous validation and was durably published to the internal queue with broker acknowledgment. It does not guarantee downstream persistence.
- State-dependent processing outcomes such as stale-event ignore, idempotent duplicate ignore, or same-timestamp conflicts happen asynchronously and are not returned in the immediate HTTP response. Internal async diagnostics such as `CONFLICTING_SAME_TIMESTAMP` are not public response codes.
- Items are reported independently, so one invalid product does not block the rest of the batch.
- If enqueue outcome is known per item and one or more items fail queue publishing, the server may return `207` with `QUEUE_PUBLISH_FAILED` on those items.
- If enqueue outcome is unknown for the request, the server should return `500+`, and the client may retry the whole request.

Entities

Response object

```

1 {
2   "requestId": "6cd77c96-118f-44f0-b7f7-0d3ab8297e6a",
3   "summary": {
4     "received": 3,
5     "accepted": 2,
6     "rejected": 1
7   },
8   "items": [ Item ]
9 }
```

Field	Type	Required	Description
requestId	String	Yes	Server-generated request identifier. Also returned in <code>X-Request-Id</code> header.
summary	Object	Yes	Batch-level counts.

items	Array	Yes	Array of item . Item-level processing results, in no guaranteed order.
-------	-------	-----	--

summary object:

Field	Type	Required	Description
received	Integer	Yes	Number of items received in the batch.
accepted	Integer	Yes	Number of items accepted and durably enqueued with broker acknowledgment.
rejected	Integer	Yes	Number of items rejected during synchronous validation.

Item

Add productss (POST/DELETE v1/products)

```
1 {
2   "productId": "SKU125",
3   "action": "create/udpate/delete",
4   "status": "rejected/accepted",
5   "code": "MISSING_REQUIRED_FIELD",
6   "message": "imageUrl is required for create"
7 }
```

Add markets (POST v1/markets)

```
1 {
2   "marketId": "ca",
3   "productId": "SKU125",
4   "status": "rejected/accepted",
5   "code": "MISSING_REQUIRED_FIELD",
6   "message": "updatedAt is required"
7 }
```

Field	Type	Required	Description
status	String	Yes	accepted or rejected .
code	String	Conditional	Present for rejected items.
message	String	Conditional	Human-readable explanation for the item result.

productId	String	Conditional	Present for Add productss items and Add markets items when available.
marketId	String	Conditional	Present for markets items.
action	String	Conditional	Present for products items when available.
updatedAt	String	Conditional	Echoed when useful for troubleshooting.

Per-event item identity rules:

- products item results should include at minimum productId
- markets item results should include at minimum marketId and productId
- If an item fails before productId can be parsed, the server may omit it and return only code and message

Response Codes

Status	Meaning	When
200	All accepted	Every item passed validation and was durably enqueued
207	Partial	Some items were rejected or failed queue publishing with known per-item outcome
400	Payload error	Malformed JSON, wrong eventName , missing required top-level fields
409	Conflict	Add products request languageCode does not match the stored site default and languageChanged is not true
413	Too large	Body larger than 10 MB or more than 1,000 items

429	Rate limited	Rate limit exceeded
500+	Server error	Retry with backoff

Error Codes

Code	Level	Description
INVALID_PAYLOAD	400	Malformed JSON, wrong <code>eventName</code> , missing <code>languageCode</code> for <code>Add products</code>
LANGUAGE_CODE_MISMATCH	409	Request <code>languageCode</code> does not match the stored site default. Set <code>languageChanged: true</code> to confirm the change.
MISSING_productId	207	<code>productId</code> is missing
MISSING_ACTION	207	<code>Add products</code> action missing or invalid
MISSING_REQUIRED_FIELD	207	Required field missing
INVALID_TYPE	207	Wrong type
INVALID_URL	207	Invalid absolute URL
INVALID_CURRENCY	207	Invalid ISO 4217 code
INVALID_CATEGORIES	207	Empty categories or missing category <code>id</code> and <code>name</code>
INVALID_PRICE	207	<code>oldPrice</code> <= <code>price</code> when both numeric fields are present in the same submitted item
INVALID_updatedDate	207	<code>updatedAt</code> is present but not a valid RFC3339 / ISO 8601 UTC timestamp
INVALID_NULL_FIELD	207	Field cannot be set to <code>null</code> in this context
EMPTY_MARKET_PATCH	207	<code>Add markets</code> item contains no market fields to change
DUPLICATE_KEY_IN_BATCH	207	Duplicate logical key appears more than once in the same request

QUEUE_PUBLISH_FAILED	207	Item passed validation but failed durable queue publishing with known per-item outcome
BATCH_SIZE_EXCEEDED	413	More than 1,000 items
PAYLOAD_TOO_LARGE	413	Request body larger than 10 MB
RATE_LIMITED	429	Rate limit exceeded